Technical Architecture & Expected Integrations Specification (Multi-Store Facilitator & Outsourced Delivery)

1. System Architecture Overview

The platform is architected as a **Zero-Inventory Multi-Store Marketplace Facilitator** built on the *WooCommerce REST API v3 (`/wp-json/wc/v3/`) + High-Performance Order Storage (HPOS) **pattern, serving** 6 connected interfaces** across Web and Native Mobile Apps (`Flutter`):

1. **Client Web Storefront (`Tab 1`)**: High-readability Light/Dark adaptive UI (`Plus Jakarta Sans` + `Cairo`) with Category pills, Partner Store filter, 3-step guided checkout, and floating 💬 **Support Chat Drawer**.

2. **Client My Orders & Tracking Hub (`Tab 2`)**: Dedicated order management portal featuring active vs. historical closed filters, 5-stage milestone stepper, Bosta waybill manifests, and printable Egyptian tax invoices.

3. **iOS & Android Mobile App Simulator (`Tab 3`)**: Mobile shopping simulator with Apple Pay / Meeza integration and live Firebase Push Notification stream.

4. **Partner Store Portal (`Tab 4`)**: Interactive **1-Click Store Switcher Cards**, top-priority **Incoming Sub-Orders to Pack** table, and Store-Owned Inventory manager.

5. **Outsourced Delivery Hub (`Tab 5`)**: Direct **Partner Store Pickup → Client Drop-off** dispatch board with a **Guided Next-Step Pipeline Button** (`Step 1: Store Packed` → `Step 2: Bosta Picked Up` → `Step 3: Out for Delivery` → `Step 4: Delivered` → `Step 5: Settle Cash`).

6. **Facilitator Control Tower (`Tab 6`)**: Organized into **5 focused sub-tabs** (`Commercial Rules & Fees` , `Customer Support Chat Desk` , `Orders, Deliveries & Settlements` , `All-Stores Inventory, Stores & Categories` , `Webhooks Inspector` , plus `Show All`) for clutter-free 360° administration.

```
flowchart TB
    subgraph Touchpoints["6 Role-Based Interfaces (Web + Flutter iOS/Android)"]
        ClientWeb["1. Client Web Storefront (Category & Store Filter)"]
        ClientOrders["2. My Orders & Tracking (Active & Closed History)"]
        ClientApp["3. iOS & Android Flutter App (Apple Pay & FCM Push)"]
        StorePortal["4. Partner Store Portal (Store Inventory & Packaging)"]
        DeliveryHub["5. Outsourced Delivery Hub (Store Pickup -> Client Dropoff)"]
        AdminConsole["6. Facilitator Admin (Commercial Rules & Support Desk)"]
    end

    subgraph CoreEngine["Core Marketplace Facilitator Engine (Port 4000)"]
        REST["WooCommerce v3 REST API (/wp-json/wc/v3/*)"]
        SplitEngine["Multi-Store Cart & Commission Split Calculator"]
        ManifestSplitter["Per-Store Outsourced Delivery Manifest Generator"]
        OrderEnricher["Order Lifecycle & 5-Stage Stepper Pipeline Enricher"]
        StockSaga["Store Inventory Reservation & 48h Fawry Auto-Release"]
        WebhookAuth["HMAC-SHA512 Webhook Security Verifier"]
        DB[("HPOS SQLite DB (wc_categories, wc_stores, wc_products, wc_orders, wc_deliveries, wc_settlements)")]
    end

    subgraph ExternalIntegrations["Egyptian Fintech & Outsourced 3PL Integrations (Port 4001 Mockup)"]
        PaymobSplit["Paymob Accept / Split API (Cards, Meeza, Apple Pay, Wallets)"]
        Fawry["FawryPay Reference Code API (48h Store Stock Hold)"]
        ValU["valU BNPL API (6/12/24m Installments)"]
        WhatsApp["WhatsApp Cloud API (COD OTP 4455 Verification)"]
        Bosta3PL["Bosta Outsourced 3PL API (Multi-Store Pickup -> Client Delivery)"]
    end

    ClientWeb --> REST
    ClientOrders --> REST
    ClientApp --> REST
    StorePortal --> REST
    DeliveryHub --> REST
    AdminConsole --> REST

    REST --> SplitEngine
    REST --> ManifestSplitter
    REST --> OrderEnricher
    REST --> StockSaga
    REST --> WebhookAuth
    SplitEngine --> DB
    ManifestSplitter --> DB
    OrderEnricher --> DB
    StockSaga --> DB

    ManifestSplitter <-->|"Multi-Store Split"| PaymobSplit
    ManifestSplitter <-->|"48h Kiosk Hold"| Fawry
    ManifestSplitter <-->|"Installment Plan"| ValU
    ManifestSplitter <-->|"COD OTP"| WhatsApp
    ManifestSplitter <-->|"Per-Store Pickup AWB"| Bosta3PL

    PaymobSplit -->|"HMAC-SHA512 Webhook"| WebhookAuth
    Fawry -->|"HMAC-SHA512 Webhook"| WebhookAuth
    ValU -->|"HMAC-SHA512 Webhook"| WebhookAuth
    WhatsApp -->|"HMAC-SHA512 Webhook"| WebhookAuth
    Bosta3PL -->|"HMAC-SHA512 Webhook"| WebhookAuth
```

2. Multi-Store & Outsourced Delivery Database Schema (ER Diagram)

```
erDiagram
    WC_CATEGORIES ||--o{ WC_PRODUCTS : "categorizes"
    WC_STORES ||--o{ WC_PRODUCTS : "owns stock"
    WC_STORES ||--o{ WC_DELIVERIES : "pickup origin"
    WC_ORDERS ||--|{ WC_ORDER_ITEMS : "contains"
    WC_ORDERS ||--|{ WC_DELIVERIES : "splits into per-store shipments"

    WC_CATEGORIES {
        int id PK
        string slug UK
        string name_en
        string name_ar
        string icon_emoji
    }

    WC_STORES {
        int id PK
        string code UK
        string name_en
        string name_ar
        string governorate
        string pickup_address
        string contact_phone
        float commission_pct "Per-store or Global Commission %"
        float vat_pct "Per-store or Global Egypt VAT % (default 14%)"
        float delivery_fee_override_egp "Optional Custom/Free Delivery Fee"
        float cod_fee_override_egp "Per-store or Global COD Fee"
        string settlement_cycle "T+1 | INSTANT | WEEKLY"
    }

    WC_PLATFORM_SETTINGS {
        int id PK
        float default_commission_pct
        float default_vat_pct
        string vat_mode "INCLUSIVE | ADDITIVE"
        float extra_store_pickup_fee_egp
        float cod_fee_egp
        string default_settlement_cycle
    }

    WC_SUPPORT_TICKETS ||--|{ WC_SUPPORT_MESSAGES : "contains thread"

    WC_SUPPORT_TICKETS {
        int id PK
        string ticket_number UK "SUP-xxxxx"
        string customer_name
        string customer_phone
        string order_number
        int store_id FK
        string subject
        string status "OPEN | RESOLVED"
    }

    WC_SUPPORT_MESSAGES {
        int id PK
        int ticket_id FK
        string sender_role "CLIENT | ADMIN"
        string sender_name
        string message_text
        string created_at
    }

    WC_PRODUCTS {
        int id PK
        string sku UK
        int store_id FK
        int category_id FK
        string name_en
        string name_ar
        float price_egp
        int stock_quantity
        int reserved_quantity
    }

    WC_ORDERS {
        int id PK
        string order_number UK
        string channel "web | ios_app | android_app"
        string customer_name
        string customer_phone
```

```
        string area_street
        string governorate
        string payment_method
        string payment_status
        string order_status
        float subtotal_egp
        float vat_amount_egp
        float shipping_fee_egp
        float cod_fee_egp
        float total_egp
        float platform_commission_egp
        float net_stores_payout_egp
        string created_at
        string updated_at
    }

    WC_DELIVERIES ||--o| WC_SETTLEMENTS : "settles_into"

    WC_DELIVERIES {
        int id PK
        int order_id FK
        int store_id FK
        string store_name
        string pickup_governorate
        string pickup_address
        string customer_name
        string dropoff_governorate
        string dropoff_address
        string items_summary
        float store_subtotal_egp
        float platform_commission_egp
        float net_store_payout_egp
        float cod_to_collect_egp
        string packaging_status "AWAITING_STORE_PACKAGING | READY_FOR_COURIER_PICKUP"
        string bosta_awb "BST-xxxxxxx"
        string courier_status "PICKUP_SCHEDULED | PICKED_UP_FROM_STORE | OUT_FOR_DELIVERY | DELIVERED"
        string settlement_status "UNSETTLED | ELIGIBLE_FOR_SETTLEMENT | SETTLED_TO_STORE"
        string settlement_ref "STL-xxxxxx"
    }

    WC_SETTLEMENTS {
        int id PK
        string settlement_ref UK "STL-xxxxxx"
        int delivery_id FK
        int order_id FK
        int store_id FK
        string store_name
        string payment_method
        string bosta_awb
        float gross_amount_egp
        float platform_commission_egp
        float net_store_payout_egp
        float cod_collected_egp
        string payout_rail
        string created_at
    }
```

3. REST API Contracts & Extended Facilitator Endpoints

HTTP Method & Endpoint	Actor / Consumer	Description		
`GET /wp-json/wc/v3/orders/track`	Client Orders Hub	Returns unified tracking payload: `{success, query, total_count, active_count, closed_count, orders}` enriched with 5-stage timelines, Bosta AWBs, and closed booleans.		
`GET /wp-json/wc/v3/orders`	Client Orders Hub, Admin	Enhanced with query parameters `?scope=active	closed	all&phone=...&order_number=...&q=...` to filter active vs. historical closed orders.
`GET /wp-json/wc/v3/orders/{id_or_number}`	Client Orders Hub, Admin	Fetches single order enriched with full 5-stage milestone progression, itemized sub-manifests, and printable invoice payload.		
`GET /wp-json/wc/v3/products/categories`	Client Web, Mobile App, Store Portal	Lists all **Product Categories** with bilingual names, icons, and active SKU counts.		
`POST /wp-json/wc/v3/products/categories`	Facilitator Admin	Creates a new **Product Category** (`slug` , `name_en` , `name_ar` , `icon_emoji`).		
`GET /wp-json/wc/v3/stores`	All Interfaces	Lists all onboarded **Partner Stores**, pickup addresses, commission rates (`%`), and live payout ledgers (`gross_sales_egp` , `commission_egp` , `net_payout_egp` , `settled_payout_egp` , `pending_payout_egp`).		
`POST /wp-json/wc/v3/stores`	Facilitator Admin	Onboards a new **Partner Store** with its Governorate pickup origin and commission rate.		
`POST /wp-json/wc/v3/products`	Partner Store Portal	Publishes a new store-owned **Product** (`store_id` , `category_id` , `sku` , `name_en` , `name_ar` , `price_egp` , `stock_quantity`).		
`POST /wp-json/wc/v3/products/{id}/stock`	Partner Store Portal	Allows a Partner Store to increment/decrement its own `stock_quantity` while protecting `reserved_quantity` .		
`POST /wp-json/wc/v3/cart/calculate`	Client Web, Mobile App	Calculates multi-store cart totals, promo discounts (`EGYPT10` , `RAMADAN200`), multi-origin Bosta delivery fees, **Platform Commission**, and **Net Store Payouts**.		
`POST /wp-json/wc/v3/orders`	Client Web, Mobile App	Reserves stock in each Partner Store, splits items by `store_id` into `wc_deliveries` manifests, and initializes the chosen Egyptian payment gateway on Port `4001` .		
`GET /wp-json/wc/v3/deliveries`	Outsourced Delivery	Lists all **Store-to-Client Outsourced Delivery Manifests**, including pickup origin, client		

HTTP Method & Endpoint	Actor / Consumer	Description	
	Hub, Store Portal	destination, packaging state, Bosta AWB status, and `settlement_status`.	
`POST /wp-json/wc/v3/deliveries/{id}/store-ready`	Partner Store Portal	Marks a store sub-order `READY_FOR_COURIER_PICKUP` and fires a push notification to the Outsourced Delivery Hub and Client App.	
`POST /wp-json/wc/v3/deliveries/{id}/settle`	Store Portal, Delivery Hub, Admin	Triggers **Cash Settlement** via Port `4001` (`/mock/settlement/v1/payout`), firing an HMAC-verified `SETTLEMENT.COMPLETED` webhook to `/wp-json/wc/v3/webhooks/settlement` and recording `STL-xxxxxx` in `wc_settlements`.	
`POST /wp-json/wc/v3/settlements/settle-bulk`	Facilitator Admin	Bulk-settles all `DELIVERED` sub-orders across **ALL Partner Stores (`store_id: "all"`)** or a **Specific Store (`store_id`)**.	
`GET /wp-json/wc/v3/settlements`	Facilitator Admin	Lists all completed **Cash Settlement & Commission Realization** transactions (`wc_settlements`).	
`GET & POST /wp-json/wc/v3/settings/commercial-rules`	Facilitator Admin	Reads or updates **Commission %**, **Egypt VAT %**, **VAT Mode (`INCLUSIVE` \	`ADDITIVE`), Delivery Fee Override (`EGP`), Extra Pickup Fee (`EGP`), COD Fee (`EGP`), and Settlement Cycle** for `target_scope: "all"` or a specific `store_id`.
`POST /wp-json/wc/v3/governorates/update`	Facilitator Admin	Updates Bosta 3PL `shipping_egp` and `sla_days` for a single Governorate or `"ALL"` Governorates.	
`GET & POST /wp-json/wc/v3/support/tickets`	Client Storefront, Facilitator Admin	Lists all customer support chat threads (`SUP-xxxx`) or opens a new support chat from the Client Storefront.	
`POST /wp-json/wc/v3/support/tickets/{id}/reply`	Client Storefront, Facilitator Admin	Appends a live `CLIENT` or `ADMIN` message to a support thread and dispatches a push notification.	
`POST /wp-json/wc/v3/support/tickets/{id}/status`	Facilitator Admin	Marks a support ticket `RESOLVED` or `OPEN`.	
`POST /wp-json/wc/v3/mock-proxy`	All Simulator Actions	Same-origin proxy on Port `4000` that forwards simulator triggers (`path` , `payload`) to Port `4001` (`http://127.0.0.1:4001`), preventing cross-origin blocks and IPv6 localhost resolution lag.	

4. Client Order Monitoring, Tracking & Tax Invoicing Architecture

4.1 Order Classification & Progress Calculation Engine (`enrich_order_dict`)

The backend HPOS layer dynamically annotates raw database records with rich delivery and lifecycle metadata before serving any order endpoint:

1. **Active vs. Closed Partitioning:**

- An order is classified as `is_closed = True` if its `order_status` is in `('completed', 'completed-settled', 'cancelled', 'refunded')` OR its `payment_status == 'settled'`.
- Otherwise, the order is marked `is_closed = False` (Active in Packaging, Courier Transit, or Drop-off).

2. **5-Stage Progress Stepper Algorithm:**

- `Step 1 (Order Placed):` Marked completed upon initial checkout insertion.
- `Step 2 (Payment Verified & Split):` Completed if payment status is `paid` or `settled` (or in-progress/pending).
- `Step 3 (Store Packaged):` Completed if all fulfilling partner stores have marked their parcels `READY_FOR_COURIER_PICKUP`.
- `Step 4 (Out for Delivery - Bosta 3PL):` Completed/Active if Bosta couriers have picked up parcels or parcels are out for delivery.
- `Step 5 (Delivered & Settled):` 100% completed once all deliveries reach `DELIVERED` and financial settlement `SETTLED_TO_STORE` is achieved.

3. **Multi-Store Manifest Transparency:**

- Each delivery sub-record exposes `store_name`, `pickup_governorate`, `packaging_status`, `courier_status`, and `courier_tracking_id` (Bosta AWB).

4.2 Printable Commercial Tax Invoice Standards

In full compliance with **Egyptian Tax Authority (ETA)** standards and **E-Commerce Law No. 151/2020**:

- **Facilitator Entity:** NileConnect E-Commerce Facilitator LLC
- **Tax Registration Number:** `492-817-309`
- **Commercial Register:** `184920`
- **Auditable Itemization:** Product SKU, description, fulfilling partner store origin, quantity, unit price, Egypt VAT 14%, Bosta shipping fee, COD fee, and grand total in Egyptian Pounds (EGP).

---

5. Expected Production Integrations for Your Facilitator Model

1. **Paymob Split Payments (`POST https://accept.paymob.com/v1/intention/`):**

- Pass the `store_splits` array in the payment intention request so Paymob automatically routes `net_store_payout_egp` to each Partner Store's sub-merchant account while depositing `platform_commission_egp` into your facilitator account.

2. **Bosta Multi-Pickup API (`POST https://app.bosta.co/api/v2/deliveries`):**

- For each row in `wc_deliveries`, pass the Partner Store's `pickupAddress` (`city: d.pickup_governorate, firstLine: d.pickup_address`) and the Client's `dropOffAddress` (`city: d.dropoff_governorate, firstLine: d.dropoff_address`). Bosta's courier picks up directly from the Partner Store and delivers to the Client.

3. **FawryPay 48h Store Reservation (`POST https://atfawry.com/ECommerceWeb/Fawry/payments/charge`):**

- Holds the Partner Store's stock for 48 hours (`reserved_quantity`). If `FAWRY.EXPIRED` arrives, the stock is automatically returned to the Partner Store's available inventory.

---

6. Vercel Cloud Deployment Architecture (100% Architectural Parity with Local)

The entire multi-server environment is configured to deploy to **Vercel (`@vercel/python`)** without changing a single line of business logic:

Local Service	Local URL	Vercel Serverless Route (`vercel.json` → `api/index.py`)	Runtime Mechanism
1. Core Marketplace Facilitator Engine (`ecommerce_server.py`)	`http://localhost:4000`	`/` & `/wp-json/wc/v3/*`	Handled by `EcommerceRequestHandler` backed by SQLite in `/tmp/ecommerce_hpos.db` (auto-initialized & seeded on cold start).
2. Egyptian Fintech & Bosta 3PL Sandbox (`mock_integrations_server.py`)	`http://localhost:4001`	`/mock-dashboard` & `/mock/*`	Routed to `MockIntegrationsHandler` ; outbound API calls and inbound `HMAC-SHA512` webhooks execute in-memory via `invoke_handler_in_memory()` when `VERCEL=1` .
3. Platform Study & Liability Calculator (`study.html`)	`http://localhost:8080`	`/study`	Served directly from `study.html` on the same Vercel domain.